Sri Sivasubramaniya Nadar College of Engineering
(An Autonomous Institution, Affiliated to Anna University, Chennai)
Kalavakkam – 603 110

Department of Computer Science & Engineering

Computer Networks Laboratory

1513 – Computer Networks Lab

Assignment 3

Implementation of HTTP Client using TCP Sockets

Name:	Simiyon Vinscent Samuel L
Register Number:	3122247001062
Year / Semester:	III Year / V Semester
Course:	1513 Computer Networks Lab
Date of Submission:	22 July 2026

1. Objective

To develop a TCP socket-based HTTP client in C that accepts a URL from the user, parses the host/port/path, establishes a TCP connection with a web server, sends an HTTP GET request, retrieves the requested resource, and stores it locally. The client communicates with a Python-based HTTP server (`http.server`) hosting HTML, text, PDF and image files, and analyses the communication performance in terms of response time, download time, data size and throughput.

2. Server Algorithm (Python HTTP Server)

The assignment specification requires the server side to be a stock Python web server rather than custom C code, so that the C client can be validated against a standards-compliant HTTP server.

1. Create a directory `CN_Web_Server` and place the resources to be served inside it: `index.html`, `notes.txt`, `sample.pdf`, `image.jpg`.
2. Start Python's built-in HTTP server bound to TCP port 8000:

```
python3 -m http.server 8000
```

3. Internally, `http.server` creates a TCP listening socket, `accept()`s each client connection, parses the HTTP request line, maps the requested path to a file in the served directory, and streams back a well-formed `HTTP/1.0 200 OK` response with `Content-Type` and `Content-Length` headers followed by the file bytes (or a `404` if the resource is absent).
4. Every request/response pair is logged to standard output (source address, timestamp, request line, status code).

3. Client Algorithm

1. Prompt the user for a URL and parse it into `host`, `port` (default 80) and `path` (`parse_url()`).
2. Resolve the host name using `gethostbyname()`.
3. Create a TCP socket and `connect()` to `host:port`.
4. Build and `send()` an HTTP/1.1 `GET` request with a `Connection: close` header so the server closes the connection once the response is complete.
5. Record a start timestamp, then loop on `recv()`:
   - Record the timestamp of the first received byte (used for *response time*).
   - Parse and discard the HTTP response header (up to the blank line `\r\n\r\n`).
   - Write the remaining body bytes to a local output file named `downloaded_<resource>`.
6. Record an end timestamp once `recv()` returns 0 (server closed the connection).
7. Compute and display performance metrics: response time, download time, data size and throughput (KB/s).

4. Server Code

The server is Python's standard-library `http.server` module used as-is (no custom source required by the assignment); it is launched with the single command shown in Section 2.

5. Client Code

5.1 http_client.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>
#include <netdb.h>
#include <sys/time.h>
#include <libgen.h>

void parse_url(const char *url, char *host, int *port, char *path) {
    strcpy(path, "/");
    *port = 80;

    const char *p = url;
    if (strncmp(p, "http://", 7) == 0) {
        p += 7;
    } else if (strncmp(p, "https://", 8) == 0) {
        p += 8;
        *port = 443;
    }

    const char *slash = strchr(p, '/');
    int host_len;
    if (slash) {
        host_len = slash - p;
        strcpy(path, slash);
    } else {
        host_len = strlen(p);
    }

    char host_port[256];
    strncpy(host_port, p, host_len);
    host_port[host_len] = '\0';

    char *colon = strchr(host_port, ':');
    if (colon) {
        *colon = '\0';
        *port = atoi(colon + 1);
    }
    strcpy(host, host_port);
}

int main() {
    char url[1024];
    printf("Enter URL: ");
    if (!fgets(url, sizeof(url), stdin)) {
        return -1;
    }
    url[strcspn(url, "\r\n")] = '\0';

    char host[256];
    int port;
```

```
    char path[1024];
    parse_url(url, host, &port, path);

    printf("Client extracts:\n");
    printf("Host : %s\n", host);
    printf("Port : %d\n", port);
    printf("Path : %s\n\n", path);

    struct hostent *server = gethostbyname(host);
    if (server == NULL) {
        fprintf(stderr, "Error: No such host '%s'\n", host);
        return -1;
    }

    int sock = socket(AF_INET, SOCK_STREAM, 0);
    if (sock < 0) {
        perror("Error opening socket");
        return -1;
    }

    struct sockaddr_in serv_addr;
    memset(&serv_addr, 0, sizeof(serv_addr));
    serv_addr.sin_family = AF_INET;
    memcpy(&serv_addr.sin_addr.s_addr, server->h_addr_list[0], server->h_length
        );
    serv_addr.sin_port = htons(port);

    if (connect(sock, (struct sockaddr *)&serv_addr, sizeof(serv_addr)) < 0) {
        perror("Error connecting");
        close(sock);
        return -1;
    }

    char request[2048];
    sprintf(request,
            "GET %s HTTP/1.1\r\n"
            "Host: %s\r\n"
            "Connection: close\r\n"
            "\r\n",
            path, host);

    printf("Then sends:\n%s", request);

    char path_copy[1024];
    strcpy(path_copy, path);
    char *base = basename(path_copy);
    char out_filename[512];

    if (strcmp(base, "/") == 0 || strlen(base) == 0 || strcmp(base, ".") == 0)
        {
        strcpy(out_filename, "downloaded_index.html");
    } else {
        sprintf(out_filename, "downloaded_%s", base);
    }

    if (send(sock, request, strlen(request), 0) < 0) {
        perror("Error sending request");
        close(sock);
        return -1;
    }

    FILE *fp = fopen(out_filename, "wb");
    if (!fp) {
```

```
        perror("Error opening output file");
        close(sock);
        return -1;
    }

    char buffer[4096];
    int bytes_read;
    int first_byte = 1;
    int header_done = 0;
    char header_buf[8192];
    size_t header_len = 0;
    char last_four[4] = {0};
    size_t total_body_bytes = 0;

    struct timeval t_start, t_first, t_end;
    gettimeofday(&t_start, NULL);

    while ((bytes_read = recv(sock, buffer, sizeof(buffer), 0)) > 0) {
        if (first_byte) {
            gettimeofday(&t_first, NULL);
            first_byte = 0;
        }

        if (!header_done) {
            for (int i = 0; i < bytes_read; i++) {
                last_four[0] = last_four[1];
                last_four[1] = last_four[2];
                last_four[2] = last_four[3];
                last_four[3] = buffer[i];

                if (header_len < sizeof(header_buf) - 1) {
                    header_buf[header_len++] = buffer[i];
                }

                if (memcmp(last_four, "\r\n\r\n", 4) == 0) {
                    header_done = 1;
                    header_buf[header_len] = '\0';
                    int remaining = bytes_read - (i + 1);
                    if (remaining > 0) {
                        fwrite(buffer + i + 1, 1, remaining, fp);
                        total_body_bytes += remaining;
                    }
                    break;
                }
            }
        } else {
            fwrite(buffer, 1, bytes_read, fp);
            total_body_bytes += bytes_read;
        }
    }
    gettimeofday(&t_end, NULL);
    fclose(fp);
    close(sock);

    if (first_byte) {
        printf("Error: No data received from server.\n");
        unlink(out_filename);
        return -1;
    }

    double response_time = (t_first.tv_sec - t_start.tv_sec) * 1000.0 +
                           (t_first.tv_usec - t_start.tv_usec) / 1000.0;
    double download_time = (t_end.tv_sec - t_first.tv_sec) * 1000.0 +
```

```
                            (t_end.tv_usec - t_first.tv_usec) / 1000.0;

    double download_time_sec = download_time / 1000.0;
    double throughput = 0;
    if (download_time_sec > 0) {
        throughput = (total_body_bytes / 1024.0) / download_time_sec;
    }

    printf("\n--- Performance Metrics ---\n");
    printf("Response Time : %.3f ms\n", response_time);
    printf("Download Time : %.3f ms (%.6f seconds)\n", download_time,
        download_time_sec);
    printf("Data Size     : %zu bytes (%.2f KB)\n", total_body_bytes,
        total_body_bytes / 1024.0);
    if (download_time_sec > 0) {
        printf("Throughput    : %.2f KB/s\n", throughput);
    } else {
        printf("Throughput    : Instantaneous\n");
    }
    printf("Saved resource to: %s\n", out_filename);
    printf("---------------------------\n");

    return 0;
}
```

6. Output / Test Cases

The Python server was started inside `CN_Web_Server` on port 8000 and the C client was run four times, once for each resource type required by the assignment: an HTML page, a plain text file, a PDF document and a JPEG image.

6.1 Server-side Access Log

```
::1 - - [17/Aug/2026 10:42:49] "HEAD /index.html HTTP/1.1" 200 -
::1 - - [17/Aug/2026 10:43:48] "GET /index.html HTTP/1.1" 200 -
::ffff:127.0.0.1 - - [17/Aug/2026 10:43:55] "GET /index.html HTTP/1.1" 200 -
::ffff:127.0.0.1 - - [17/Aug/2026 10:43:55] "GET /notes.txt HTTP/1.1" 200 -
::ffff:127.0.0.1 - - [17/Aug/2026 10:43:55] "GET /sample.pdf HTTP/1.1" 200 -
::ffff:127.0.0.1 - - [17/Aug/2026 10:43:55] "GET /image.jpg HTTP/1.1" 200 -
```

6.2 Client Console Output (all four resource types)

```
=== Request: http://localhost:8000/index.html ===
Enter URL: Client extracts:
Host : localhost
Port : 8000
Path : /index.html

Then sends:
GET /index.html HTTP/1.1
Host: localhost
Connection: close


--- Performance Metrics ---
Response Time : 0.935 ms
Download Time : 0.151 ms (0.000151 seconds)
Data Size     : 168 bytes (0.16 KB)
Throughput    : 1086.51 KB/s
```

```
Saved resource to: downloaded_index.html
---------------------------

=== Request: http://localhost:8000/notes.txt ===
Enter URL: Client extracts:
Host : localhost
Port : 8000
Path : /notes.txt

Then sends:
GET /notes.txt HTTP/1.1
Host: localhost
Connection: close


--- Performance Metrics ---
Response Time : 0.568 ms
Download Time : 0.447 ms (0.000447 seconds)
Data Size     : 159 bytes (0.16 KB)
Throughput    : 347.37 KB/s
Saved resource to: downloaded_notes.txt
---------------------------

=== Request: http://localhost:8000/sample.pdf ===
Enter URL: Client extracts:
Host : localhost
Port : 8000
Path : /sample.pdf

Then sends:
GET /sample.pdf HTTP/1.1
Host: localhost
Connection: close


--- Performance Metrics ---
Response Time : 0.492 ms
Download Time : 0.668 ms (0.000668 seconds)
Data Size     : 33790 bytes (33.00 KB)
Throughput    : 49398.27 KB/s
Saved resource to: downloaded_sample.pdf
---------------------------

=== Request: http://localhost:8000/image.jpg ===
Enter URL: Client extracts:
Host : localhost
Port : 8000
Path : /image.jpg

Then sends:
GET /image.jpg HTTP/1.1
Host: localhost
Connection: close


--- Performance Metrics ---
Response Time : 0.383 ms
Download Time : 0.220 ms (0.000220 seconds)
Data Size     : 332 bytes (0.32 KB)
Throughput    : 1473.72 KB/s
Saved resource to: downloaded_image.jpg
---------------------------
```

6.3 Performance Summary

Resource	Response (ms)	Download (ms)	Size (bytes)	Throughput (KB/s)
index.html	0.935	0.151	168	1086.51
notes.txt	0.568	0.447	159	347.37
sample.pdf	0.492	0.668	33790	49398.27
image.jpg	0.383	0.220	332	1473.72

All four requests succeeded with HTTP `200 OK`, and each resource was correctly saved locally as `downloaded_<name>`, matching the size reported by the server. The PDF (largest file) shows the highest throughput because its transfer amortises the fixed per-connection overhead (TCP handshake, header parsing) over far more payload bytes than the small HTML/text/image files.

7. Learning Outcomes

- Learned to manually construct and parse HTTP/1.1 request and response messages at the socket level, including correctly detecting the header/body boundary (`\r\n\r\n`).
- Understood the role of the `Connection: close` header in enabling a client to detect end-of-response via `recv()` returning 0, without a `Content-Length`.
- Practised interoperating a custom C TCP client with a standards-compliant third-party HTTP server (Python's `http.server`), reinforcing that HTTP is a text-based application-layer protocol independent of implementation language.
- Learned to measure network application performance metrics – response time, download time, and throughput – using `gettimeofday()` at precise points in the data path.
- Observed how payload size affects effective throughput due to fixed per-request overhead.